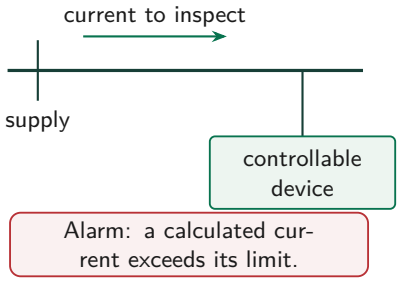
An alarm is a question: what can the operator change?



A physical question

Which permitted command reduces the loading enough, and what evidence supports it?

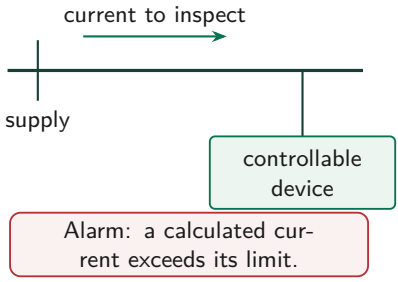
An operator needs

The candidate, the calculated consequence, the limit, and the model used.

We start with one small worked decision. Its arithmetic is invented so that every step can be checked by hand; the feeder models come later.

Teaching schematic; Graph lab G0–G5 declares the toy scope.

An alarm is a question: what can the operator change?



A physical question

Which permitted command reduces the loading enough, and what evidence supports it?

An operator needs

The candidate, the calculated consequence, the limit, and the model used.

We start with one small worked decision. Its arithmetic is invented so that every step can be checked by hand; the feeder models come later.

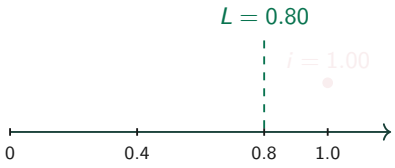
Teaching schematic; Graph lab G0–G5 declares the toy scope.

Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

Proposed input: set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$



Verdict: $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both i and L are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

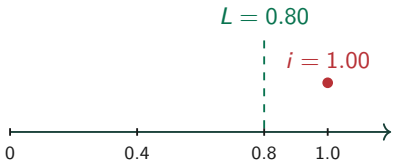
Exact response and comparison used in G2 and G4.

Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

Proposed input: set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$



Verdict: $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both i and L are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

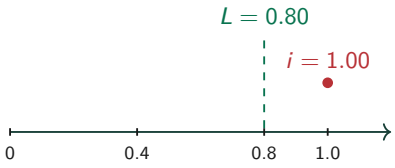
Exact response and comparison used in G2 and G4.

Work the alarm: the proposed command gives too much current

$$\underbrace{i}_{\text{toy current}} = 0.4 + 0.8\ \underbrace{u}_{\text{command}}, \qquad \underbrace{L}_{\text{toy limit}} = 0.80.$$

Proposed input: set the dimensionless command to $u = 0.75$.

$$i = 0.4 + 0.8(0.75) = 1.00.$$



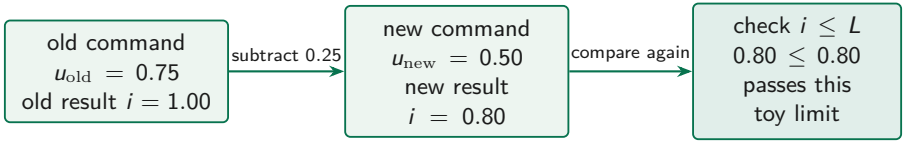
Verdict: $1.00 > 0.80$, so the candidate fails. The excess is $i - L = 0.20$.

Both i and L are normalized illustrative quantities, not amperes. A correct calculation can still return an unacceptable operating point.

Exact response and comparison used in G2 and G4.

Repair the command, then recalculate its consequence

$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$

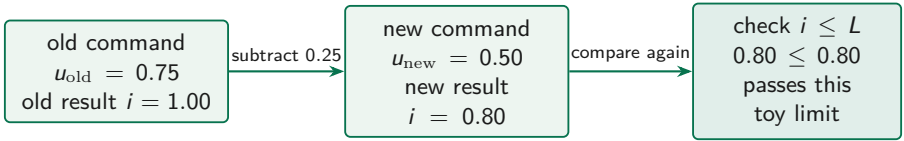


u_{old} and u_{new} are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

Repair the command, then recalculate its consequence

$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$

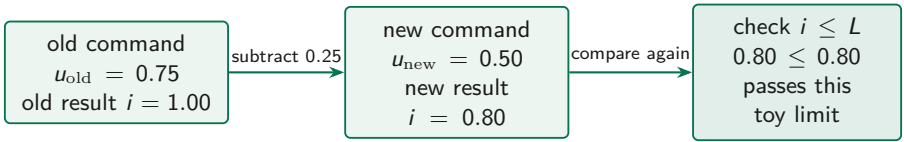


u_{old} and u_{new} are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

Repair the command, then recalculate its consequence

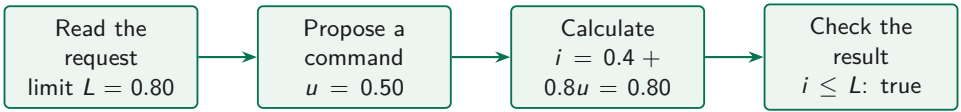
$$u_{\text{new}} = \max(0, u_{\text{old}} - 0.25) = \max(0, 0.75 - 0.25) = 0.50.$$



u_{old} and u_{new} are the commands before and after repair. The lower bound of zero prevents a negative command. The old current belongs to the old command. Reusing it would give the wrong decision about the repaired command.

Repair and simulation functions: G2.

The operator's calculation already has a workflow



A dependency

We cannot check the consequence of this command until its calculation returns.

A record

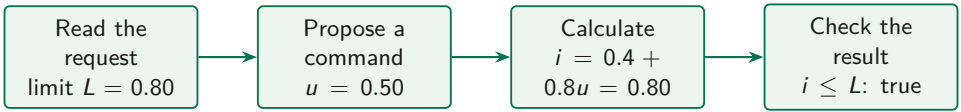
Keep the request, command, result, limit, and verdict together.

A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

The operator's calculation already has a workflow



A dependency

We cannot check the consequence of this command until its calculation returns.

A record

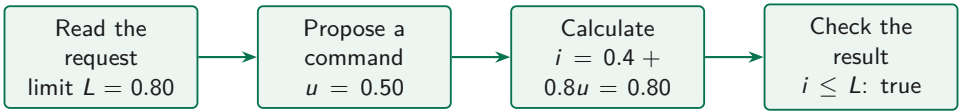
Keep the request, command, result, limit, and verdict together.

A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

The operator's calculation already has a workflow



A dependency

We cannot check the consequence of this command until its calculation returns.

A record

Keep the request, command, result, limit, and verdict together.

A next decision

A pass permits review; a failure asks for repair or a hold.

Tutorial decomposition; executable functions in G1–G3.

A reviewable recommendation is more than a pass label

Recommendation record	Value
Candidate command	$u = 0.50$
Toy current	$i = 0.80$
Declared limit	$L = 0.80$
Model scope	invented toy equation
Review identity	bound to this recommendation

Predict: Would the same acknowledgment apply after changing u?

Explanation: No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

A reviewable recommendation is more than a pass label

Recommendation record	Value
Candidate command	$u = 0.50$
Toy current	$i = 0.80$
Declared limit	$L = 0.80$
Model scope	invented toy equation
Review identity	bound to this recommendation

Predict: Would the same acknowledgment apply after changing u?

Explanation: No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

A reviewable recommendation is more than a pass label

Recommendation record	Value
Candidate command	$u = 0.50$
Toy current	$i = 0.80$
Declared limit	$L = 0.80$
Model scope	invented toy equation
Review identity	bound to this recommendation

Predict: Would the same acknowledgment apply after changing u?

Explanation: No. A changed recommendation needs its own evidence and matching readback.

The G5 exercise records a simulated acknowledgment. It neither authenticates an operator nor sends a command to equipment.

Review payload and identity test: G5; engineering handoff: L12.

Three hours: understand a process, then challenge it in code

Minutes	Question we will answer	Hands-on checkpoint
0–40	How do state, tools, and decisions connect?	G0–G5; L0–L1
40–82	Which inverter command meets the limits?	L2–L5; Llama evidence
82–92	Break	Keep the kernels available
92–119	Which permitted restoration choice serves priorities?	L6–L7
119–149	Which measurements can support a response?	L8–L9
149–180	What does the evidence justify for the operator?	L10–L12; capstone

Every exercise follows **predict, run, explain**. A fresh numerical result, an archived actual Llama reply, and a new Llama call have different labels.

Executable teaching route: graph notebook and engineering notebook.

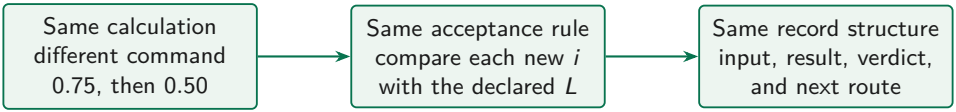
Three hours: understand a process, then challenge it in code

Minutes	Question we will answer	Hands-on checkpoint
0–40	How do state, tools, and decisions connect?	G0–G5; L0–L1
40–82	Which inverter command meets the limits?	L2–L5; Llama evidence
82–92	Break	Keep the kernels available
92–119	Which permitted restoration choice serves priorities?	L6–L7
119–149	Which measurements can support a response?	L8–L9
149–180	What does the evidence justify for the operator?	L10–L12; capstone

Every exercise follows **predict, run, explain**. A fresh numerical result, an archived actual Llama reply, and a new Llama call have different labels.

Executable teaching route: graph notebook and engineering notebook.

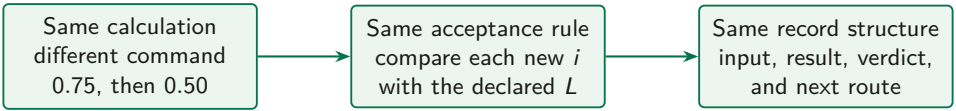
Automate the repeated work; keep its evidence visible



A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

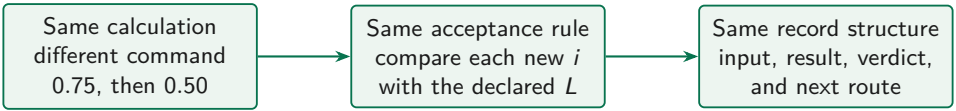
Automate the repeated work; keep its evidence visible



A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

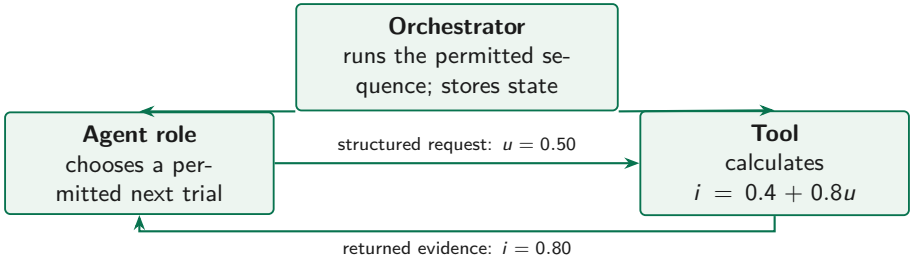
Automate the repeated work; keep its evidence visible



A conventional program can perform all three responsibilities. An agentic design must earn its value through useful choices, clearer evidence, or adaptable task handling. We will compare numerical outcome, recorded checks, and elapsed time separately. More model calls do not by themselves establish an advantage.

Matched evaluation: PowerAgentBench-SS, 2026 preprint; tutorial baseline: G4.

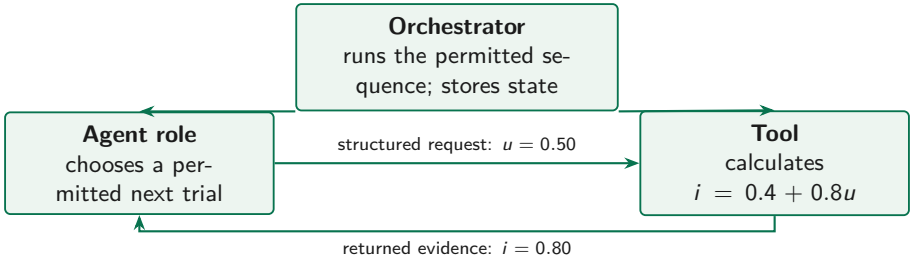
Three names for three jobs we have already seen



A role is a responsibility, not necessarily a separate model. A numerical node can operate without an LLM; several roles can use the same Llama backbone. The graph lab uses deterministic choices. The engineering workflow later adds actual bounded Llama responses.

Role-based systems: AutoGen; tutorial separation: workflow implementation.

Three names for three jobs we have already seen



A role is a responsibility, not necessarily a separate model. A numerical node can operate without an LLM; several roles can use the same Llama backbone. The graph lab uses deterministic choices. The engineering workflow later adds actual bounded Llama responses.

Role-based systems: AutoGen; tutorial separation: workflow implementation.

Llama can choose follow-up work; the tool supplies the numbers

Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

Allowed operation	Meaning
`REPAIR`	try the declared repair rule
`HOLD`	stop and report the issue

What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

This is an interface illustration, not a captured model reply. The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

Llama can choose follow-up work; the tool supplies the numbers

Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

Allowed operation	Meaning
`REPAIR`	try the declared repair rule
`HOLD`	stop and report the issue

What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

This is an interface illustration, not a captured model reply. The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

Llama can choose follow-up work; the tool supplies the numbers

Illustrative language interface

This candidate exceeds its limit. Choose an allowed repair or request a hold.

Allowed operation	Meaning
`REPAIR`	try the declared repair rule
`HOLD`	stop and report the issue

What the tool still does

Calculate the repaired consequence and compare it with the unchanged limit.

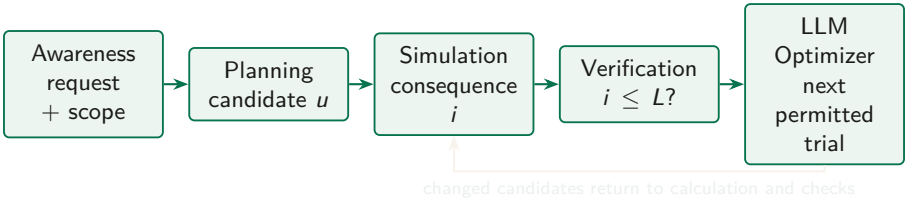
What we inspect

The chosen operation, returned result, verdict, and concise public explanation.

This is an interface illustration, not a captured model reply. The toy uses fixed routing; later voltage examples show actual Llama outputs and latency.

Observe–act feedback: ReAct; actual replies: L3, L10, L11.

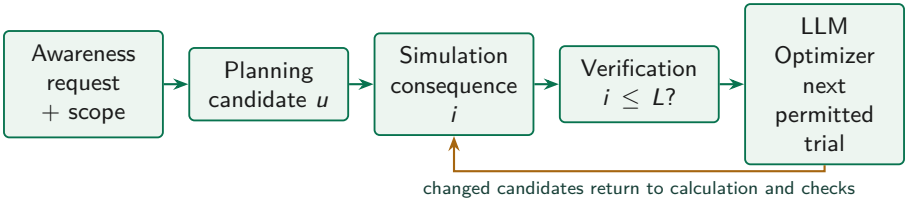
All five proposed stages have a concrete return value



Human review receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

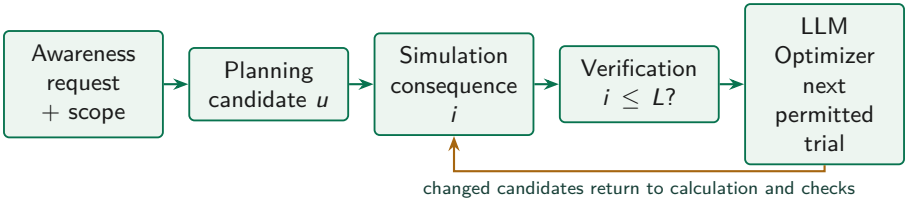
All five proposed stages have a concrete return value



Human review receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

All five proposed stages have a concrete return value



Human review receives the selected candidate, its evidence, alternatives, and unresolved assumptions. Planning uses a suitable numerical method for each case. The LLM Optimizer selects follow-up work; a new choice is a trial, not an already verified action.

All five stages and recorded events: executed tutorial trace.

The workflow reuses numerical methods; it does not replace them

Case	Numerical work	What the workflow coordinates
Voltage support	MPC planning; SAC continuous-action proposal	Compare candidates, simulate, check, select a bounded trial
Restoration	Mixed-integer service allocation	Preserve priorities and feasible connectivity while evaluating options
Cyber response	State estimation, residual tests, constrained response	Distinguish suspicious evidence, supported action, and a hold

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

The workflow reuses numerical methods; it does not replace them

Case	Numerical work	What the workflow coordinates
Voltage support	MPC planning; SAC continuous-action proposal	Compare candidates, simulate, check, select a bounded trial
Restoration	Mixed-integer service allocation	Preserve priorities and feasible connectivity while evaluating options
Cyber response	State estimation, residual tests, constrained response	Distinguish suspicious evidence, supported action, and a hold

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

The workflow reuses numerical methods; it does not replace them

Case	Numerical work	What the workflow coordinates
Voltage support	MPC planning; SAC continuous-action proposal	Compare candidates, simulate, check, select a bounded trial
Restoration	Mixed-integer service allocation	Preserve priorities and feasible connectivity while evaluating options
Cyber response	State estimation, residual tests, constrained response	Distinguish suspicious evidence, supported action, and a hold

A model-based method predicts behavior with an explicit model. A model-free policy learns an action rule from experience; training may still use a simulator. Every candidate faces declared checks. A language-model explanation is not a numerical certificate.

Methods: classroom tools; MPC; SAC.

State is the record that keeps this decision coherent

Before the repair

`command`	0.75
`current`	1.00
`limit`	0.80
`passed`	false
`attempts`	0

After repair and recalculation

`command`	0.50
`current`	0.80
`limit`	0.80
`passed`	true
`attempts`	1

s_t denotes the full record before workflow step t. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

State is the record that keeps this decision coherent

Before the repair		After repair and recalculation	
`command`	0.75	`command`	0.50
`current`	1.00	`current`	0.80
`limit`	0.80	`limit`	0.80
`passed`	false	`passed`	true
`attempts`	0	`attempts`	1

s_t denotes the full record before workflow step t. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

State is the record that keeps this decision coherent

Before the repair	
`command`	0.75
`current`	1.00
`limit`	0.80
`passed`	false
`attempts`	0

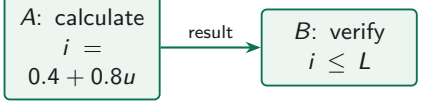
After repair and recalculation	
`command`	0.50
`current`	0.80
`limit`	0.80
`passed`	true
`attempts`	1

s_t denotes the full record before workflow step t. A node reads that record and returns field updates; unchanged fields remain available. The limit remains 0.80. Repairing the command does not authorize relaxing its acceptance criterion.

Exact fields and updates: G1–G2.

Two graphs answer different questions

Workflow graph: what must run first?



Nodes are operations; $A \rightarrow B$ means that B requires a result from A.

Electrical graph: what is connected?

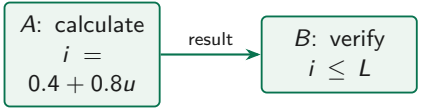
Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. A, B: operations; s: shared record; $r(s)$: a rule returning the next operation label. The earlier u, i, L remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

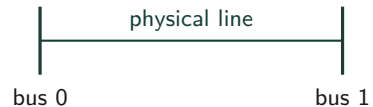
Two graphs answer different questions

Workflow graph: what must run first?



Nodes are operations; $A \rightarrow B$ means that B requires a result from A.

Electrical graph: what is connected?



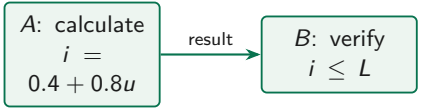
Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. A, B: operations; s: shared record; $r(s)$: a rule returning the next operation label. The earlier u, i, L remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

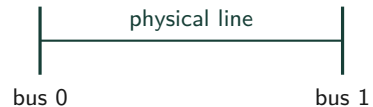
Two graphs answer different questions

Workflow graph: what must run first?



Nodes are operations; $A \rightarrow B$ means that B requires a result from A.

Electrical graph: what is connected?

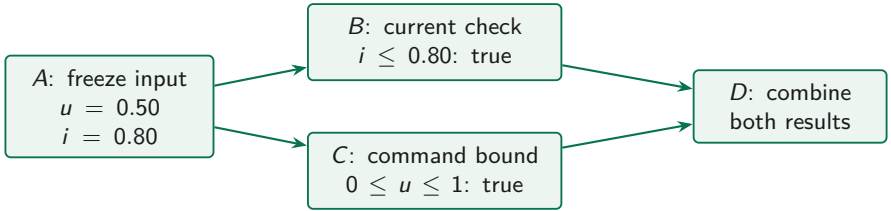


Nodes are buses; edges are lines or other network connections. Their equations describe electrical behavior.

A tool may solve the electrical graph while the workflow graph controls when that tool runs. The two drawings describe different systems. A, B: operations; s: shared record; $r(s)$: a rule returning the next operation label. The earlier u, i, L remain the toy command, consequence, and limit.

LangGraph graph API; OpenDSS documentation.

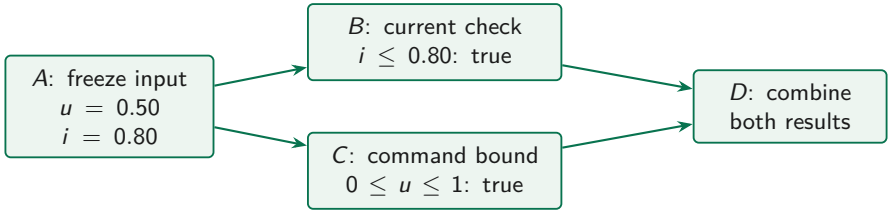
A DAG can branch two checks and join their evidence



$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; E contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

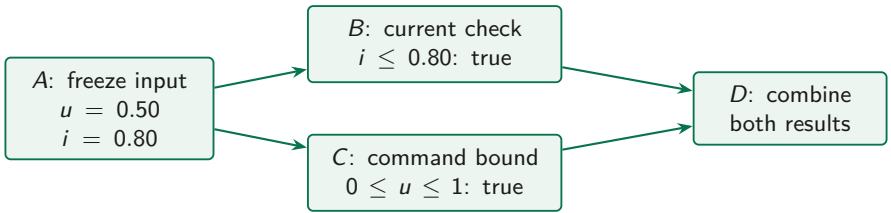
A DAG can branch two checks and join their evidence



$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; E contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

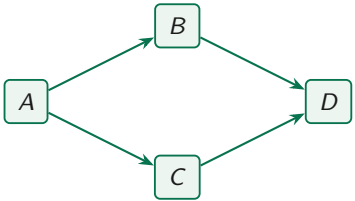
A DAG can branch two checks and join their evidence



$\mathcal{G} = (N, E)$: $N = \{A, B, C, D\}$ is the operation set; E contains the directed dependency arrows. **Directed**: arrows have direction. **Acyclic**: no directed path returns to its start. This is a DAG. This teaching example is not the G-lab topology. The join requires both results for the same frozen input.

DAG ordering: NetworkX; illustrative checks use the declared toy variables.

A valid schedule puts both checks before the join

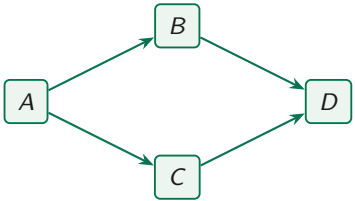


Candidate schedule	Valid?
A, B, C, D	Yes
A, C, B, D	Yes
A, B, D, C	No: D lacks C

A: freeze input; B: current check; C: command bound; D: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

A valid schedule puts both checks before the join

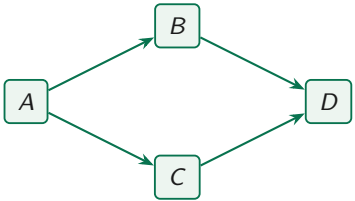


Candidate schedule	Valid?
A, B, C, D	Yes
A, C, B, D	Yes
A, B, D, C	No: D lacks C

A: freeze input; B: current check; C: command bound; D: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

A valid schedule puts both checks before the join

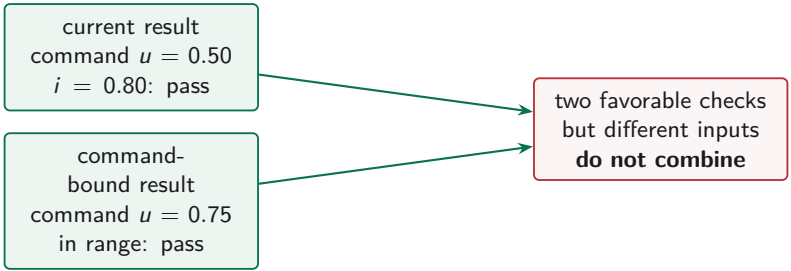


Candidate schedule	Valid?
A, B, C, D	Yes
A, C, B, D	Yes
A, B, D, C	No: D lacks C

A: freeze input; B: current check; C: command bound; D: combine evidence. The two middle checks may exchange order. That does not establish measured speedup: execution resources and shared tool state still matter.

Topological ordering: NetworkX.

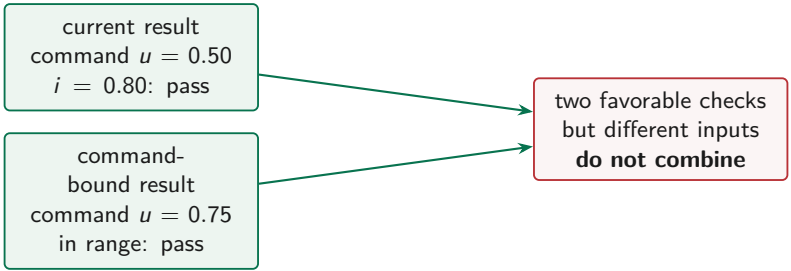
The join must match the candidate, not merely collect two ticks



Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

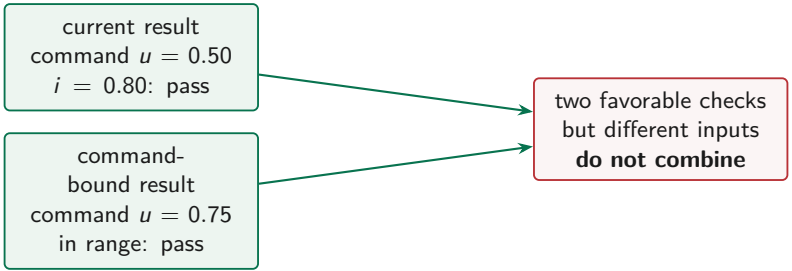
The join must match the candidate, not merely collect two ticks



Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

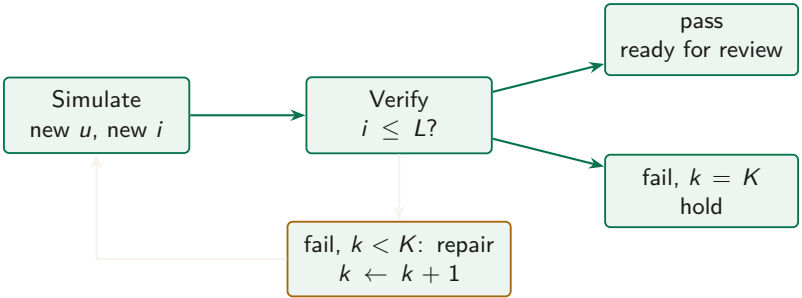
The join must match the candidate, not merely collect two ticks



Both calculations can be individually correct. Together they do not establish the stated checks for one common candidate. Bind each result to the request revision and command. In L1, changing either makes the old binding fail.

Exact request/candidate binding exercise: L1.

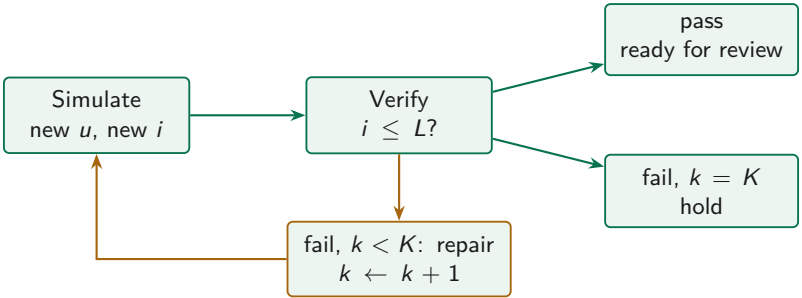
A repair loop is a cycle, so give it a stopping rule



k: completed repairs; K: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

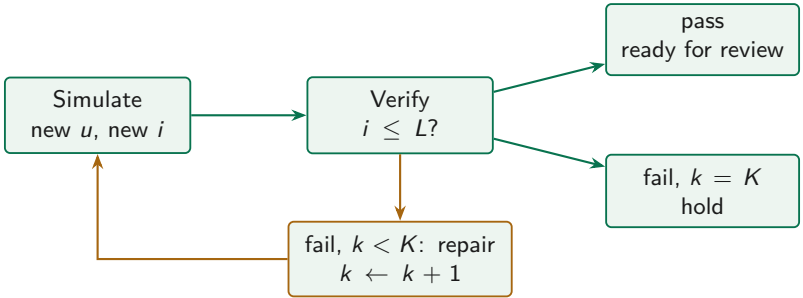
A repair loop is a cycle, so give it a stopping rule



k: completed repairs; K: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

A repair loop is a cycle, so give it a stopping rule



k: completed repairs; K: maximum repairs; both are nonnegative integers. The backward edge makes the program cyclic, not a DAG. The budget bounds attempts if tool calls return. It guarantees neither a feasible command nor a tool timeout.

Exact bounded policy: G2–G4; LangGraph.

Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here s is the current record; $r(s)$ returns one next-node label. k is the completed repair count and K is the repair budget.

Predict: A failed check has $k = 1$ and $K = 1$. Which route is permitted?

Explanation: `hold`. The budget is exhausted; equality does not permit another repair.
Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here s is the current record; $r(s)$ returns one next-node label. k is the completed repair count and K is the repair budget.

Predict: A failed check has $k = 1$ and $K = 1$. Which route is permitted?

Explanation: `hold`. The budget is exhausted; equality does not permit another repair.

Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

Route from the verdict and the remaining budget

$$r(s) = \begin{cases} \texttt{report}, & \text{if passed is true,} \\ \texttt{repair}, & \text{if passed is false and } k < K, \\ \texttt{hold}, & \text{otherwise.} \end{cases}$$

Here s is the current record; $r(s)$ returns one next-node label. k is the completed repair count and K is the repair budget.

Predict: A failed check has $k = 1$ and $K = 1$. Which route is permitted?

Explanation: `hold`. The budget is exhausted; equality does not permit another repair.
Verification computes the verdict. Routing decides what to do with that verdict.

The implemented route is four Python lines in G2.

A state schema names the facts each node may need

Fields	Our example	Purpose
`command`, `limit`	0.75, 0.80	identify the trial and unchanged criterion
`current`, `passed`	1.00, `False`	store the latest calculation and comparison
`attempts`, `max_repairs`	0, 2	decide whether another repair is allowed
`trace`	ordered event list	explain how the current record was reached

$s_{t+1} = \mathrm{merge}(s_t, \Delta s_t)$. s_t: record before step t; Δs_t: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

A state schema names the facts each node may need

Fields	Our example	Purpose
`command`, `limit`	0.75, 0.80	identify the trial and unchanged criterion
`current`, `passed`	1.00, `False`	store the latest calculation and comparison
`attempts`, `max_repairs`	0, 2	decide whether another repair is allowed
`trace`	ordered event list	explain how the current record was reached

$s_{t+1} = \text{merge}(s_t, \Delta s_t)$. s_t: record before step t; Δs_t: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

A state schema names the facts each node may need

Fields	Our example	Purpose
`command`, `limit`	0.75, 0.80	identify the trial and unchanged criterion
`current`, `passed`	1.00, `False`	store the latest calculation and comparison
`attempts`, `max_repairs`	0, 2	decide whether another repair is allowed
`trace`	ordered event list	explain how the current record was reached

$s_{t+1} = \text{merge}(s_t, \Delta s_t)$. s_t: record before step t; Δs_t: returned field updates, not arithmetic subtraction. Python `TypedDict` describes expected keys and types. The tutorial's `initial_state` function performs explicit input-value checks.

Python TypedDict; validated inputs in graph-lab helper.

A reducer states whether new information replaces or appends

Replace the current result

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

Append the event history

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b \qquad R_{\text{trace}}(a, b) = a + b$$

R is a reducer; a is the stored value and b the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

A reducer states whether new information replaces or appends

Replace the current result

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

Append the event history

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b$$

$$R_{\text{trace}}(a, b) = a + b$$

R is a reducer; a is the stored value and b the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

A reducer states whether new information replaces or appends

Replace the current result

Old current 1.00; new current 0.80. Store 0.80 for the repaired command.

Append the event history

Old list [simulate]; new list [verify]. Store [simulate, verify].

$$R_{\text{current}}(a, b) = b \qquad R_{\text{trace}}(a, b) = a + b$$

R is a reducer; a is the stored value and b the returned update. In the right-hand expression, $+$ means list concatenation. Preserve history without pretending an old current is the result for a new command. Parallel writes also need a meaningful merge rule.

List reducer in G1; LangGraph reducers.

Three starting states test three distinct paths

Start (u, L, K)	First result	After one repair	Expected outcome
$(0.25, 0.80, 2)$	$i = 0.60$	not needed	ready immediately
$(0.75, 0.80, 2)$	$i = 1.00$	$u = 0.50,\ i = 0.80$	ready after repair
$(0.90, 0.50, 1)$	$i = 1.12$	$u = 0.65,\ i = 0.92$	hold: still above limit

u: command; L: current limit; K: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

Three starting states test three distinct paths

Start (u, L, K)	First result	After one repair	Expected outcome
$(0.25, 0.80, 2)$	$i = 0.60$	not needed	ready immediately
$(0.75, 0.80, 2)$	$i = 1.00$	$u = 0.50,\ i = 0.80$	ready after repair
$(0.90, 0.50, 1)$	$i = 1.12$	$u = 0.65,\ i = 0.92$	hold: still above limit

u: command; L: current limit; K: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

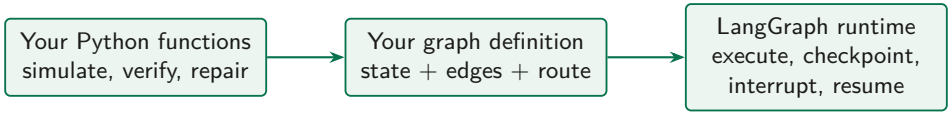
Three starting states test three distinct paths

Start (u, L, K)	First result	After one repair	Expected outcome
$(0.25, 0.80, 2)$	$i = 0.60$	not needed	ready immediately
$(0.75, 0.80, 2)$	$i = 1.00$	$u = 0.50,\ i = 0.80$	ready after repair
$(0.90, 0.50, 1)$	$i = 1.12$	$u = 0.65,\ i = 0.92$	hold: still above limit

u: command; L: current limit; K: repair budget; $i = 0.4 + 0.8u$. Repair subtracts 0.25, bounded below by zero. The final case is useful: the process stops correctly even though it does not find an acceptable command within its budget.

These are the exact three cases executed in G4.

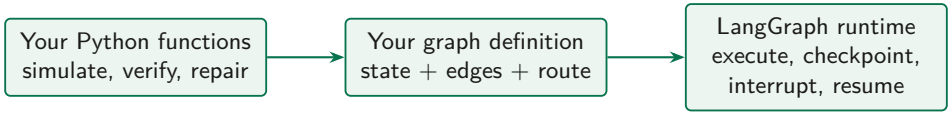
LangGraph runs the stateful process we have already specified



The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

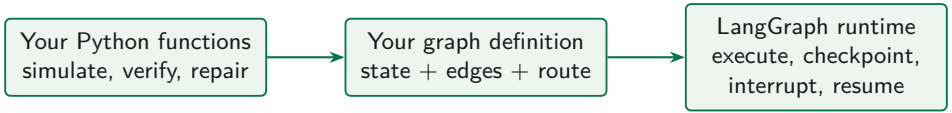
LangGraph runs the stateful process we have already specified



The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

LangGraph runs the stateful process we have already specified



The functions still contain the arithmetic. The route still contains the stopping policy. LangGraph makes their execution structure explicit. Our G notebook uses real LangGraph without an LLM or electrical simulator. The engineering notebooks remain custom Python; this is not a claim that they were migrated to LangGraph.

LangGraph overview; executed G notebook.

G0: import the runtime and record what this experiment is

Notebook G0 Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

G0: import the runtime and record what this experiment is

Notebook G0 Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

G0: import the runtime and record what this experiment is

Notebook G0 Run the full setup cell in `Graph_Workflow_Lab.ipynb`.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

The printed versions identify the environment. The last line identifies the scope: rule-driven toy execution, with no model call or electrical validation. After dependencies are installed, this exercise needs neither Ollama nor OpenDSS.

Exact notebook lines 5–13: G0; LangGraph.

G1: the schema is our before-and-after table in Python

Notebook G1 Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

G1: the schema is our before-and-after table in Python

Notebook G1 Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

G1: the schema is our before-and-after table in Python

Notebook G1 Run the full cell; connect each field to the worked example.

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

Trial and gate

`command`, `limit`, `current`, and `passed` record the comparison.

Execution history

`attempts` counts repairs. `trace` appends event lists using `add`.

`TypedDict` describes the schema; input validation is performed separately.

Exact state schema: G1; TypedDict.

G2: simulation returns a number; verification returns a verdict

Notebook G2 Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

Simulation update

For $u = 0.75$, return `current = 1.00` and an event.

Verification update

Compare 1.00 with 0.80; return `passed = False` and another event.

The 10^{-12} code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

G2: simulation returns a number; verification returns a verdict

Notebook G2 Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

Simulation update

For $u = 0.75$, return `current = 1.00` and an event.

Verification update

Compare 1.00 with 0.80; return `passed = False` and another event.

The 10^{-12} code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

G2: simulation returns a number; verification returns a verdict

Notebook G2 Run the whole cell; these are its first two node functions.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

Simulation update

For $u = 0.75$, return `current = 1.00` and an event.

Verification update

Compare 1.00 with 0.80; return `passed = False` and another event.

The 10^{-12} code tolerance addresses floating-point comparison in this toy; it is not an engineering operating margin.

Exact functions: G2, lines 1–7.

G2: repair changes the command; routing checks the budget

Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

G2: repair changes the command; routing checks the budget

Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

G2: repair changes the command; routing checks the budget

Change the command and count the repair

```
def repair(s):
    return {"command": max(0., s["command"] - 0.25),
            "attempts": s["attempts"] + 1, "trace": [
     "repair: -0.25"]}
```

Choose the permitted next node

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

The command drops by 0.25, bounded below by zero; `attempts` increases by one. Neither line changes the limit. A pass returns `report`. Otherwise, the route returns `repair` only while the counter is smaller than its maximum.

Exact repair and route: G2, lines 9–11 and 19–22.

G3: the backward edge makes every repair face a fresh calculation

Notebook G3 Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

G3: the backward edge makes every repair face a fresh calculation

Notebook G3 Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

G3: the backward edge makes every repair face a fresh calculation

Notebook G3 Run the cell and inspect the compiled node list.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

`simulate` → `verify` always calculates before checking. The conditional edge reads the label returned by `route`. `repair` → `simulate` is the return loop. Compilation checks structure; it does not prove the response law or its physical assumptions.

Exact graph wiring: G3; LangGraph construction.

G4: predict, run, then explain one complete trace

Notebook G4 Run unchanged, then use a separate scratch cell for a new budget.

Case	Final command	Final current	Status
Already admissible	0.25	0.60	`TOY_READY`
One repair	0.50	0.80	`TOY_READY`
Budget exhausted	0.65	0.92	`TOY_HOLD`

Repair trace: simulate 1.00 → verify false → repair → simulate 0.80 → verify true → report.

Predict: What if the exhausted case gets four repairs instead of one?

Explanation: It reaches $u = 0$, hence $i = 0.40 \leq 0.50$, and can report ready.

Exact outputs, trace assertion, and cases: G4.

G4: predict, run, then explain one complete trace

Notebook G4 Run unchanged, then use a separate scratch cell for a new budget.

Case	Final command	Final current	Status
Already admissible	0.25	0.60	`TOY_READY`
One repair	0.50	0.80	`TOY_READY`
Budget exhausted	0.65	0.92	`TOY_HOLD`

Repair trace: simulate 1.00 $\rightarrow$ verify false $\rightarrow$ repair $\rightarrow$ simulate 0.80 $\rightarrow$ verify true $\rightarrow$ report.

Predict: What if the exhausted case gets four repairs instead of one?

Explanation: It reaches $u = 0$, hence $i = 0.40 \leq 0.50$, and can report ready.

Exact outputs, trace assertion, and cases: G4.

G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

Controlled comparison

Same initial states, response law, repair rule, verification, and route.

Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

Controlled comparison

Same initial states, response law, repair rule, verification, and route.

Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

G4: hold the functions fixed and compare the supervisor

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

Controlled comparison

Same initial states, response law, repair rule, verification, and route.

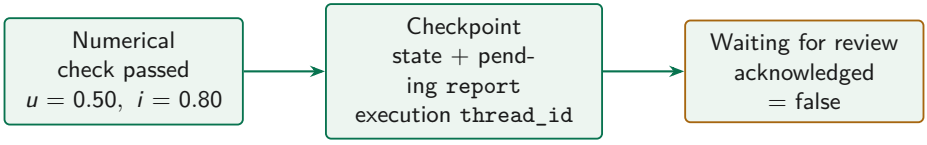
Observed result

All three commands, statuses, and event histories match the direct-Python baseline.

The graph did not improve this toy control result. Its explicit structure supports inspection and the pause/resume mechanism we will test next.

Executable baseline equality assertion: G4, lines 15–19.

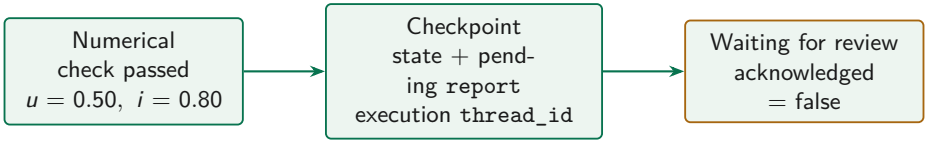
A checkpoint preserves the record and the next unfinished step



A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

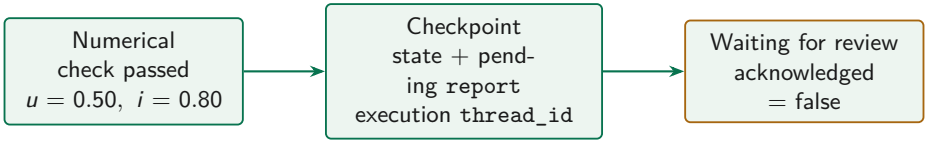
A checkpoint preserves the record and the next unfinished step



A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

A checkpoint preserves the record and the next unfinished step



A checkpoint is a saved execution record. A thread identifier associates a later response with that execution; it is not an operator credential. The lab uses `InMemorySaver`. It supports a pause within the running kernel, not durable recovery after the process is lost.

LangGraph persistence; actual checkpoint inspection: G5.

G5: interrupt with the recommendation that needs a response

```python
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

G5: interrupt with the recommendation that needs a response

```
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

G5: interrupt with the recommendation that needs a response

```
def human_review(s):
    token = recommendation_id(s)
    reply = interrupt({"recommendation_id": token, "command": s["command"],
                       "toy_current": s["current"], "scope": "invented toy equation only"})
    accepted = (isinstance(reply, dict) and reply.get("approve") is True
                and reply.get("recommendation_id") == token)
    return {"acknowledged": accepted,
            "status": "TOY_ACKNOWLEDGED" if accepted else "TOY_HOLD",
            "trace": [f"human review: {accepted}"]}
```

The payload states the command, toy consequence, recommendation identity, and scope. A matching approval is required to record acknowledgment. On resume, the node can run again from its beginning. Do not put an uncontrolled equipment action before the interrupt.

Exact review function: G5, lines 1–9; interrupt semantics.

G5: pause and inspect before resuming

Notebook G5 For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

Expected pre-resume output: pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

G5: pause and inspect before resuming

Notebook G5 For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

Expected pre-resume output: pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

G5: pause and inspect before resuming

Notebook G5 For a visible pause, execute lines 1–17 first; then inspect the output.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

Expected pre-resume output: pending node `('report',)`; `acknowledged: False`. The payload describes the exact toy recommendation. Running the full cell also supplies a scripted response afterward. Inspect both printed stages rather than overlooking the intermediate pause.

Exact pause and snapshot: G5, lines 11–17.

G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

Matching simulated approval

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

Decline or wrong identity

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

Matching simulated approval

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

Decline or wrong identity

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

G5: resume the same execution with a matching readback

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

Matching simulated approval

Status becomes `TOY_ACKNOWLEDGED`; no node remains pending.

Decline or wrong identity

The review function returns `TOY_HOLD`; acknowledgment remains false.

This is a state-management exercise. The scripted response does not authenticate a person, and the example sends no equipment command.

Exact resume and final assertion: G5; LangGraph interrupts.

L1: a changed request must not reuse old evidence

Notebook L0–L1 Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

Same request and same command	`True`
Changed load; old evidence	`False`
Changed command; old evidence	`False`

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

L1: a changed request must not reuse old evidence

Notebook L0–L1 Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

Same request and same command	`True`
Changed load; old evidence	`False`
Changed command; old evidence	`False`

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

L1: a changed request must not reuse old evidence

Notebook L0–L1 Open `Classroom_Labs.ipynb`; run setup and the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

Same request and same command	`True`
Changed load; old evidence	`False`
Changed command; old evidence	`False`

The payload is explicitly `example_only`. Hash binding checks consistency; it does not prove physical feasibility or source authenticity.

Exact inputs and expected outputs: L1; binding implementation.

Schema, binding, and engineering checks answer different questions

Check	Question	A pass does not establish
Schema	Are required fields and values valid?	Correct interpretation of the operator's intent
Binding	Does this result belong to this candidate?	Authenticity of its source
Engineering	Do modeled limits pass?	Adequate model scope or field authorization

Predict: A solver converges at 112 A; the declared limit is 100 A. Two LLMs favor the candidate. Accept it?

Explanation: No. The current check fails by 12 A. A vote cannot raise the declared limit.

Tutorial evidence and limit checks: classroom tools; case adapters.

Schema, binding, and engineering checks answer different questions

Check	Question	A pass does not establish
Schema	Are required fields and values valid?	Correct interpretation of the operator's intent
Binding	Does this result belong to this candidate?	Authenticity of its source
Engineering	Do modeled limits pass?	Adequate model scope or field authorization

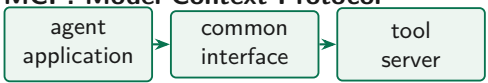
Predict: A solver converges at 112 A; the declared limit is 100 A. Two LLMs favor the candidate. Accept it?

Explanation: No. The current check fails by 12 A. A vote cannot raise the declared limit.

Tutorial evidence and limit checks: classroom tools; case adapters.

MCP connects tools; MPC computes a control plan

MCP: Model Context Protocol



It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

MPC: Model Predictive Control

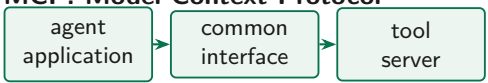
It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

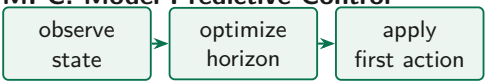
MCP connects tools; MPC computes a control plan

MCP: Model Context Protocol



It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

MPC: Model Predictive Control



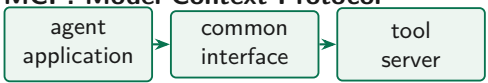
It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

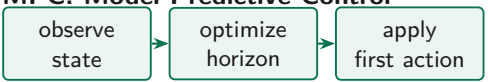
MCP connects tools; MPC computes a control plan

MCP: Model Context Protocol



It standardizes tool discovery, arguments, and results. It does not determine whether a current is acceptable.

MPC: Model Predictive Control



It uses a predictive model, objective, and constraints to plan control, then replans after the next observation.

An MPC solver could be exposed through MCP. Our classroom functions are called directly in Python; the tutorial does not require an MCP server.

MCP tools specification; Rawlings et al., MPC.

MPC plans with a model; SAC supplies a learned continuous action

Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

H: planning horizon; k: predicted step; x_k: predicted state; u_k: control; ℓ: stage cost. t: current decision time; o_t: observation; π_θ: policy with learned parameters θ. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

MPC plans with a model; SAC supplies a learned continuous action

Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

H: planning horizon; k: predicted step; x_k: predicted state; u_k: control; ℓ: stage cost. t: current decision time; o_t: observation; π_θ: policy with learned parameters θ. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

MPC plans with a model; SAC supplies a learned continuous action

Model-based planning

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to a prediction model and operating constraints. Apply the first action, observe again, and replan.

Model-free policy inference

$$u_t = \pi_\theta(o_t).$$

The trained policy maps an observation to a continuous command. Class uses saved SAC weights; it does not train a new actor.

H: planning horizon; k: predicted step; x_k: predicted state; u_k: control; ℓ: stage cost. t: current decision time; o_t: observation; π_θ: policy with learned parameters θ. **Same acceptance process:** simulate both candidates and check the same declared limits. PPO is discussed as an alternative; it is not trained here.

MPC; SAC; PPO.

Prior frameworks contribute mechanisms we can teach and test

Framework	Useful mechanism	Connection to our example
PowerAgent ecosystem	reusable engineering interfaces and workflows	expose the calculation with inspectable inputs and outputs
PowerDAG	exemplar retrieval; prerequisite advisory	detect a covered dependency problem before a tool call
Grid-Agent v3	specialized roles; sandbox validation and rollback	keep a failed trial separate from the incumbent
X-GridAgent	planning–coordination–action hierarchy	preserve task context across handoffs

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

Prior frameworks contribute mechanisms we can teach and test

Framework	Useful mechanism	Connection to our example
PowerAgent ecosystem	reusable engineering interfaces and workflows	expose the calculation with inspectable inputs and outputs
PowerDAG	exemplar retrieval; prerequisite advisory	detect a covered dependency problem before a tool call
Grid-Agent v3	specialized roles; sandbox validation and rollback	keep a failed trial separate from the incumbent
X-GridAgent	planning–coordination–action hierarchy	preserve task context across handoffs

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

Prior frameworks contribute mechanisms we can teach and test

Framework	Useful mechanism	Connection to our example
PowerAgent ecosystem	reusable engineering interfaces and workflows	expose the calculation with inspectable inputs and outputs
PowerDAG	exemplar retrieval; prerequisite advisory	detect a covered dependency problem before a tool call
Grid-Agent v3	specialized roles; sandbox validation and rollback	keep a failed trial separate from the incumbent
X-GridAgent	planning–coordination–action hierarchy	preserve task context across handoffs

PowerDAG covers selected rules and permits repeated calls after its one-shot advisory. That mechanism is not an unconditional hard safety gate. These are close prior works, not a leaderboard. Our tutorial does not claim the first multi-agent grid workflow or first use of numerical verification.

PowerAgent; PowerDAG; Grid-Agent v3; X-GridAgent.

A capability tick describes documented scope, not superiority

Work	Tools	Feedback	Checks	Review	Qualification
PowerAgent	✓	✓	△	✓	capabilities depend on workflow
PowerDAG	✓	✓	△	NR	selected rules; one-shot advisory
Grid-Agent v3	✓	✓	✓	△	sandbox; deployment oversight proposed
X-GridAgent	✓	✓	△	△	solver checks; prompt refinement
PowerAgentBench-SS	✓	✓	✓	△	DC benchmark; hidden revalidation
Tutorial	✓	✓	✓	✓	declared models; local readback

✓: documented; △: qualified or proposed; NR: not reported in the inspected source. Benchmark work evaluates agents; it is not itself a controller. Common scenarios, tools, budgets, and outcome definitions are needed before claiming an advantage.

Source-by-source qualifications: research comparison; PowerAgentBench-SS.

A capability tick describes documented scope, not superiority

Work	Tools	Feedback	Checks	Review	Qualification
PowerAgent	✓	✓	△	✓	capabilities depend on workflow
PowerDAG	✓	✓	△	NR	selected rules; one-shot advisory
Grid-Agent v3	✓	✓	✓	△	sandbox; deployment oversight proposed
X-GridAgent	✓	✓	△	△	solver checks; prompt refinement
PowerAgentBench-SS	✓	✓	✓	△	DC benchmark; hidden revalidation
Tutorial	✓	✓	✓	✓	declared models; local readback

✓: documented; △: qualified or proposed; NR: not reported in the inspected source. Benchmark work evaluates agents; it is not itself a controller. Common scenarios, tools, budgets, and outcome definitions are needed before claiming an advantage.

Source-by-source qualifications: research comparison; PowerAgentBench-SS.

Compare benefit in three columns, with the same experiment

Numerical outcome

Objective value, served load, constraint violations, and model scope.

Workflow outcome

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

Cost of the decision

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

Compare benefit in three columns, with the same experiment

Numerical outcome

Objective value, served load, constraint violations, and model scope.

Workflow outcome

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

Cost of the decision

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

Compare benefit in three columns, with the same experiment

Numerical outcome

Objective value, served load, constraint violations, and model scope.

Workflow outcome

Stale evidence rejected, required checks recorded, and recommendation identity preserved.

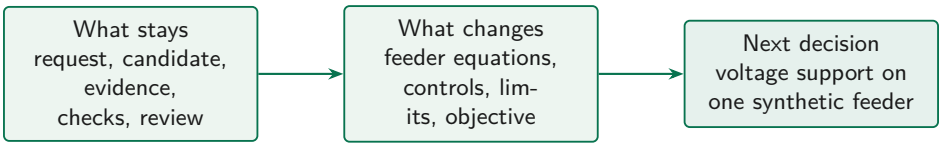
Cost of the decision

Solver time, model-call time, number of trials, and training cost reported separately.

The G baseline already gives a useful null result: the commands tie exactly. That result demonstrates graph mechanics, not better physical control. For the engineering comparison, hold the scenario, available controls, objective, constraints, and trial budget fixed where a paired claim is made.

Tutorial matched baseline: G4; evaluation design: PowerAgentBench-SS.

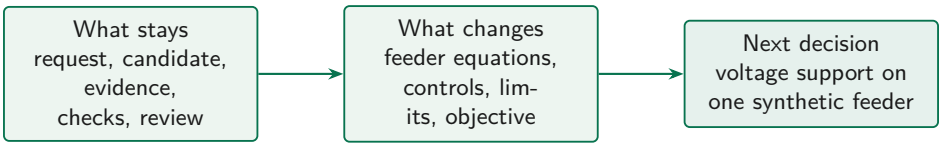
Keep the process; replace the toy law with electrical models



MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

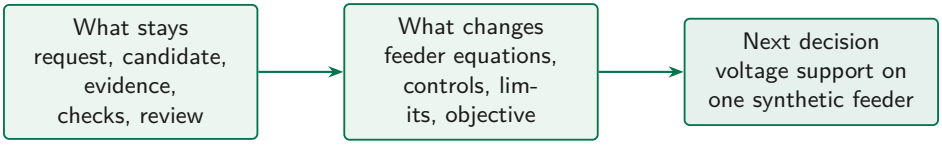
Keep the process; replace the toy law with electrical models



MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.

Keep the process; replace the toy law with electrical models



MPC and SAC will propose inverter commands. Three-phase LinDistFlow and OpenDSS will expose differences in modeled consequences and scope. Llama's bounded choices, public explanations, and latency will be inspected alongside numerical baselines. A favorable explanation never substitutes for a fresh check.

Voltage experiments and exact outputs: L2–L5; response traces: five-stage archive.